COMPUTER ORGANIZATION & ARCHITECTURE (COA)

Module 2.1 & Module 2.2 — Complete 1-Hour Exam-Cracker Master Guide

Prepared for S.Y. B.Tech (KJSCE / Somaiya Vidyavihar University | Course Code: 116U01C303)

Every Concept Explained in Simple Vocabulary + Kitchen/Body Analogies + Zero-Wrap Diagrams + 100% PYQ & Syllabus Coverage

How to Beat Module 2 in 60 Minutes (Ordered by Highest Exam Probability First)

Do not study randomly. Follow the exact 60-minute stopwatch sequence below so you lock in the guaranteed 10-mark and 15-mark Somaiya exam questions in the first 45 minutes:

Minute Window	Section	Exact Exam Deliverable You Lock In	Marks
Min 00 – 10	**Section 1**	8086 16-bit Flag Register diagram (`O-D-I-T S-Z-x-A-x-P-x-C`) + Bit-string decoding (*MSE 2025 Q3 & 2023*)	**10 Marks**
Min 10 – 20	**Section 2**	All 8 Addressing Modes + Effective Address (EA) + Instruction Formats (`0,1,2,3-Addr`) + 5 Instructions of 8086 (*ESE 2023 Q2Bi & Q1v*)	**10–15 Marks**
Min 20 – 30	**Section 3**	Instruction Cycle (`Fetch, Indirect, Execute, Interrupt` RTL micro-ops) + `ADD R1, X` + Memory Read/Write Sequencing (*ESE 2023 Q1ii*)	**10–15 Marks**
Min 30 – 37	**Section 4**	Internal Structure of CPU diagram + User-Visible vs Control/Status Registers (*ESE 2023 Q2A OR*)	**10 Marks**
Min 37 – 47	**Section 5**	Need of Control Unit + Wilkes' Microprogrammed Control Unit diagram + Hardwired vs Microprogrammed (*ESE 2023 Q2Bii & Q5i*)	**15 Marks**
Min 47 – 54	**Section 6**	Horizontal vs Vertical Micro-instructions + Sequencing & Execution + 5 Applications of Microprogramming + Nano-programming (*ESE 2023 Q2Aii*)	**5–10 Marks**
Min 54 – 60	**Sec 7 & 8**	CISC vs RISC comparison + RISC Pipelining + Hazards (*ESE 2023 Q2Ai & Q5v*) + 3-Problem Interleaved Boss Test & Quadruple Audit	**5–10 Marks**

Contents

1 Priority #1: 8086 Flag Register & Bit Decoding (Mod 2.1 | 10 Marks) 3

1.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 3
1.2 Stage 2: Zero-Wrap Diagram & 10-Second Positional Hook . . . 3
1.3 Stage 3: Exact Exam-Sheet Write-Up (10 Marks) . . . 4
1.4 Stage 4: Worked → Faded → Cold Drill Ladder . . . 5
1.5 Stage 5: 5-Line Survival Recall Card . . . 5

2 Priority #2: Addressing Modes, Effective Address, Formats & 8086 Instructions (Mod 2.1 | 10–15 Marks) 7

2.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 7
2.2 Stage 2: 3-Second Visual Classifier ("Shakal Pehchano") & Decision Tree . . . 7
2.3 Stage 3: Exact Exam-Sheet Write-Up (Approved M2 Q1 & ESE 2023) . . . 8
2.4 Stage 4: Worked → Cold Drill (EA & Modes) . . . 9

2.5 Stage 5: Contrast Table & 5-Line Survival Recall Card . . . 10

3 Priority #3: Instruction Cycle, Interrupts, Micro-Operations & Memory Read/Write (Mod 2.1 | 10–15 Marks) 11

3.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 11
3.2 Stage 2: Zero-Wrap Instruction Cycle with Interrupts Diagram . . . 11
3.3 Stage 3: Exact Exam-Sheet Write-Up (Approved M2 Q3, Q4, Q5 & ESE 2023 Q1ii) . . . 12
3.4 Stage 4: Active Retrieval Drills (L2 & L3) . . . 13

4 Priority #4: CPU Architecture, Internal Structure & Register Organization (Mod 2.1 | 10 Marks) 14

4.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 14
4.2 Stage 2: Zero-Wrap Internal Structure of the CPU Diagram (10M) . . . 14
4.3 Stage 3: Exact Exam-Sheet Write-Up (10 Marks) . . . 15
4.4 Stage 4: 5-Line Survival Recall Card . . . 15

5 Priority #5: Need of Control Unit, Wilkes' Design & Hardwired vs Microprogrammed (Mod 2.2 | 15 Marks) 17

5.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 17
5.2 Stage 2: Zero-Wrap Wilkes' Micro-Programmed Control Unit Diagram (10M) . . . 17
5.3 Stage 3: Exact Exam-Sheet Write-Up (10M + 5M) . . . 18
5.4 Stage 4: 5-Line Survival Recall Card . . . 19

6 Priority #6: Horizontal vs Vertical, Sequencing, Applications & Nano-Programming (Mod 2.2 | 5–10 Marks) 20

6.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 20
6.2 Stage 2: Zero-Wrap Diagrams (Horizontal/Vertical & Nano-Programming) . . . 20
6.3 Stage 3: Exact Exam-Sheet Write-Up (5M / 10M) . . . 21

7 Priority #7: CISC vs RISC, RISC Pipelining & Hazards (Mod 2.2 | 5–10 Marks) 23

7.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy) . . . 23
7.2 Stage 2: Zero-Wrap RISC Pipelining Timing Diagram (ESE 2023 Q2Ai — 5M) . . . 23
7.3 Stage 3: Exact Exam-Sheet Write-Up (CISC vs RISC, Pipelining & Hazards) . . . 23

8 Interleaved Boss Test & Quadruple-Check Syllabus/PYQ Audit 25

8.1 Part 1: R5 Interleaved Boss Set (3 Mixed Exam Problems) . . . 25
8.2 Part 2: Quadruple-Check Syllabus & PYQ Audit Matrix (Nothing Missed!) . . . 25

1. Priority #1: 8086 Flag Register & Bit Decoding (Mod 2.1 | 10 Marks)

Mode: D (Decode) | **Target:** Approved M2 Q2; Somaiya MSE 2025 Q3 (10M) & 2023 (10M) | **Time:** 10 Mins

1.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
When the ALU finishes subtracting two numbers (say $5 - 5 = 0$), the answer 0 goes into a register. But how does a conditional jump instruction (JZ — Jump if Zero) in the very next clock cycle know whether the last answer was zero, negative, or overflowed?

— COVER BELOW AND WRITE ON PAPER FIRST —

[ANSWER]: The ALU automatically flips 1-bit indicator switches inside a special 16-bit dashboard called the **Flag Register** (also called the Program Status Word / PSW).

Left Board (Dr. Berg's Diagnostic Whiteboard: Symptom → Root Cause → Antidote)

- **Symptom:** Your CPU calculates results, but cannot make IF-ELSE decisions or control how it steps through strings and interrupts.
- **Root Cause:** Raw arithmetic outputs disappear as soon as the next calculation runs, leaving no memory of *what just happened* (was there a carry? was it zero? did the sign break?).
- **Hardware Antidote:** A **16-bit Flag Register** wired directly to the ALU output:
  - **6 Status Flags (Conditional Flags):** Automatic warning lights turned ON (1) or OFF (0) by the ALU after math/logic operations (CF, PF, AF, ZF, SF, OF).
  - **3 Control Flags:** Manual switches flipped by the programmer to steer how the CPU behaves (TF, IF, DF).
  - **7 Undefined Bits (U or X):** Empty dashboard slots left unused for future chips.

Right Board (Gaurav Thakur's Domino Smartboard & Kitchen/Body Map)

Imagine your car's dashboard while driving:

1. **Domino 1:** Your foot presses the pedal (ALU performs ADD or SUB).
2. **Domino 2:** Instantly, dashboard warning lights blink on their own — *"Low Fuel"*, *"Engine Overheat"* (**6 Status Flags** tell you the condition of the math result).
3. **Domino 3:** Meanwhile, your hand flips the **Cruise Control** or **Reverse Gear** switch (**3 Control Flags** tell the car *how* to drive next).
4. **Kitchen/Body Map:** ALU = Muscles doing the chopping; **Flag Register = Nervous system's instant reflex scoreboard** reporting whether the chop was clean, overflowed the bowl, or hit zero.
5. **Where the analogy breaks:** Car dashboard lights only warn a human driver, whereas 8086 Flag bits are physical flip-flops directly wired into the Control Unit's branching circuits to trigger hardware jumps automatically.

1.2 Stage 2: Zero-Wrap Diagram & 10-Second Positional Hook

The 10-Second Positional Mnemonic (Left Bit 15 → Right Bit 0):

- First 4 bits (15 . . 12) are always unused (U/X).
- From Bit 11 down to Bit 0, chant: **"O - D - I - T | S - Z - x - A | x - P - x - C"**
- *Memory Sentence:* "**O**ur **D**og **I**s **T**ired; **S**it **Z**ero-space **A**way, **P**lease **C**ome!"

```
8086 16-BIT FLAG REGISTER (Bit 15 on Far Left -> Bit 0 on Far Right)
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
| 15 | 14 | 13 | 12 | 11 | 10 |  9 |  8 |  7 |  6 |  5 |  4 |  3 |  2 |  1 |  0 |
| U/X| U/X| U/X| U/X| OF | DF | IF | TF | SF | ZF | U/X| AF | U/X| PF | U/X| CF |
```

```
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
  Unused (15..12)     O - D - I - T       S - Z - x - A       x - P - x - C

Inline Quick Legend:
 * Status Flags  (6): CF(0), PF(2), AF(4), ZF(6), SF(7), OF(11)
 * Control Flags (3): TF(8), IF(9), DF(10)
 * Undefined     (7): Bits 1, 3, 5, 12, 13, 14, 15 (Marked as U in slides, X in exams)
```

60-Second Exam Drawing Order:

1. Draw a horizontal bar split into **16 boxes**, numbering **15 on the far LEFT** down to **0 on the far RIGHT** (never number left-to-right!).
2. Fill U (or X) in boxes `15, 14, 13, 12`, then `5, 3, 1`.
3. Fill `OF, DF, IF, TF` in `11, 10, 9, 8` and `SF, ZF, U, AF, U, PF, U, CF` in `7..0`.

1.3 Stage 3: Exact Exam-Sheet Write-Up (10 Marks)

Bit Pos	Flag Name & Category	Meaning When Set (1) vs Reset (0)
Bit 0	**CF — Carry Flag** *(Status)*	**1:** Unsigned carry out (or borrow in) at the MSB (most significant bit). **0:** No carry out.
Bit 2	**PF — Parity Flag** *(Status)*	**1:** Result has an **even number of 1s** (Even Parity). **0:** Result has an odd number of 1s.
Bit 4	**AF — Auxiliary Carry** *(Status)*	**1:** Carry from **Bit 3 to Bit 4** (lower nibble to upper nibble, used in BCD math). **0:** No nibble carry.
Bit 6	**ZF — Zero Flag** *(Status)*	**1:** Arithmetic/logic result is completely **zero (0)**. **0:** Result is non-zero.
Bit 7	**SF — Sign Flag** *(Status)*	Copies the MSB of the result. **1:** Result is **negative**. **0:** Result is **positive**.
Bit 8	**TF — Trap Flag** *(Control)*	**1:** CPU enters **single-step debugging mode** (executes 1 instruction at a time). **0:** Normal execution.
Bit 9	**IF — Interrupt Flag** *(Control)*	**1:** CPU **enables** maskable external interrupts (INTR). **0:** Disables maskable interrupts.
Bit 10	**DF — Direction Flag** *(Control)*	**1:** String instructions **auto-decrement** address (high to low). **0: Auto-increment** (low to high).
Bit 11	**OF — Overflow Flag** *(Status)*	**1: Signed arithmetic overflow** occurred (result too large for signed bits). **0:** No signed overflow.
Bits 1, 3, 5, 12–15	**U / X — Undefined Bits**	Unused / Reserved / Don't-care bits in 8086 (always write this row for the 10th mark!).

[NOTES ALERT]: 1. Parity Flag Conflict: Your slide diagram on `Module 2 p. 22` states `PF = 1` for **Even Parity**, whereas text on `p. 26` reverses it. Always write: *"As per the 8086 register diagram, $PF = 1$ when the result has an even number of 1 bits (even parity), and 0 for odd parity."*
2. X vs U Symbol: Your notes diagram prints U (Undefined), while the Somaiya MSE 2025 paper prints X and awards **1 full mark for explaining X**. Always write: *"X / U represents Undefined (unused/reserved) bits at positions 1, 3, 5, and 12 to 15."*

[SILLY-DOUBT CLEARED] 1. What is the difference between Carry Flag (CF) and Overflow Flag (OF)?
Answer: CF watches **unsigned** math (like a car odometer rolling past 9999), while OF watches **signed 2's complement** math (like adding two positive numbers and accidentally getting a negative sign bit because the answer could not fit!).

[SILLY-DOUBT CLEARED] 2. Why do we need Auxiliary Carry (AF) at Bit 3 → Bit 4 when we already have CF?
Answer: BCD (Binary Coded Decimal) stores each decimal digit (0–9) in **4 bits (1 nibble)**. Therefore, BCD addition needs to know whether the first 4 bits (`Bits 0..3`) carried over into `Bit 4`!

1.4 Stage 4: Worked → Faded → Cold Drill Ladder

L1 Worked Example (Actual Somaiya MSE 2025 Q3 — 10 Marks)

Question: Following is the 16-bit flag register for 8086. Explain and decode each flag bit as per their position (9M for 9 flags + 1M for explaining X):

X X X X 1 0 1 1 0 1 X 1 X 0 X 1

Pre-Flight Recipe: Number the 16 symbols from **15 (left)** to **0 (right)** and align O-D-I-T | S-Z-x-A | x-P-x-C under bits 11..0:

Bit Position	Bit	Flag	Decoded Meaning ("Because" Justification for Full Marks)
15, 14, 13, 12	X,X,X,X	**Undefined**	Unused / reserved don't-care bits in 8086 architecture **(part of 1M for X)**.
11	1	**OF = 1**	**Signed Overflow occurred** because OF at bit 11 is set to 1 **(1M)**.
10	0	**DF = 0**	**String operations auto-increment** (low to high address) because DF at bit 10 is 0 **(1M)**.
9	1	**IF = 1**	**Maskable external interrupts are enabled** because IF at bit 9 is 1 **(1M)**.
8	1	**TF = 1**	**CPU is in single-step trap mode** for debugging because TF at bit 8 is 1 **(1M)**.
7	0	**SF = 0**	**Result is positive** (MSB of result is 0) because SF at bit 7 is 0 **(1M)**.
6	1	**ZF = 1**	**Result of the operation is zero (0)** because ZF at bit 6 is 1 **(1M)**.
5	X	**Undefined**	Unused bit at position 5.
4	1	**AF = 1**	**Auxiliary carry generated from Bit 3 to Bit 4** (BCD nibble carry) because AF is 1 **(1M)**.
3	X	**Undefined**	Unused bit at position 3.
2	0	**PF = 0**	**Result has odd parity** (odd number of 1s) because PF at bit 2 is 0 **(1M)**.
1	X	**Undefined**	Unused bit at position 1.
0	1	**CF = 1**	**Carry out (or borrow) generated from MSB** because CF at bit 0 is 1 **(1M)**.

L2 Faded Drill (Fill the [???] blanks on paper first)

Question: Decode the 8086 flag string X X X X 0 1 1 0 1 0 X 0 X 1 X 0:

- **Bit 11 (OF = 0):** No signed overflow. | **Bit 10 (DF = 1):** [???]
- **Bit 9 (IF = 1):** Interrupts enabled. | **Bit 8 (TF = 0):** Normal execution (single-step disabled).
- **Bit 7 (SF = 1):** [???] | **Bit 6 (ZF = 0):** Result is non-zero.
- **Bit 4 (AF = 0):** No nibble carry. | **Bit 2 (PF = 1):** [???] | **Bit 0 (CF = 0):** No MSB carry.

— COVER BELOW AND WRITE ON PAPER FIRST —

[COMPLETED FADED LINES]:

- **Bit 10 (DF = 1):** String instructions **auto-decrement** (process from higher address to lower address).
- **Bit 7 (SF = 1):** Result is **negative** (MSB of the result is 1).
- **Bit 2 (PF = 1):** Result has **even parity** (even number of 1 bits).

1.5 Stage 5: 5-Line Survival Recall Card

1. Which bit number is on the far left and far right of the 8086 flag register? → **Bit 15 (left); Bit 0 (right).**

2. Write the 12-bit chant from Bit 11 down to Bit 0. → **O(11)-D(10)-I(9)-T(8) | S(7)-Z(6)-x(5)-A(4) | x(3)-P(2)-x(1)-C(0).**
3. Name the 3 Control Flags and their bit numbers. → **TF (Bit 8), IF (Bit 9), DF (Bit 10).**
4. Does PF = 1 mean even parity or odd parity according to your p. 22 diagram? → **Even parity (even number of 1s).**
5. Which 7 bit positions are X / U (Undefined)? → **Bits 1, 3, 5, 12, 13, 14, 15.**

2. Priority #2: Addressing Modes, Effective Address, Formats & 8086 Instructions (Mod 2.1 | 10–15 Marks)

Mode: D (Decode) | **Target:** Approved M2 Q1 + ESE 2023 Q2Bi (10M) + ESE 2023 Q1v (5M) + Syllabus 2.1 Formats | **Time:** 10 Mins

2.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
When an instruction says ADD AX, ..., why doesn't the CPU always just put the raw number inside the instruction? Why invent 8 different ways ("Addressing Modes") to point to data?

— COVER BELOW AND WRITE ON PAPER FIRST —

[ANSWER]: Putting a fixed constant inside an instruction works for $x + 5$, but cannot loop through a 1,000-element array or follow a pointer! Addressing modes give the CPU flexible ways to calculate the **Effective Address (EA)** of data dynamically at runtime.

Left Board (Dr. Berg's Diagnostic Whiteboard)

- **Symptom:** If every instruction could only hold a fixed memory address, looping through an array of 100 numbers would require writing 100 separate instructions!
- **Root Cause:** Fixed constants cannot move dynamically during a loop.
- **Hardware Antidote: Addressing Modes** + **Effective Address (EA)**.
  - An **Addressing Mode** is the rule telling the CPU *where and how* to find the operand (data).
  - The **Effective Address (EA)** is the final memory offset (distance from the start of the memory segment) calculated using registers and constants inside the instruction.

Right Board (Gaurav Thakur's Domino Chain & Delivery Analogy)

Imagine ordering food to your friend's flat:

1. **Immediate (#30):** I hand you the chocolate bar right now inside my sentence. Zero travel!
2. **Register (BX):** It is already in your left hand (BX). Zero memory trip!
3. **Direct ([1234h]):** Go straight to Flat #1234.
4. **Register Indirect ([SI]):** Ask the watchman (SI); he holds the paper slip with the flat number.
5. **Based / Indexed ([BX+5] / [SI+5]):** Start at Building BX (or Floor SI) and walk 5 doors ahead!
6. **Based Indexed with Displacement ([BX][SI+20]):** Building BX + Floor SI + Room 20!
7. **Where the analogy breaks:** Flats are physical rooms, whereas an 8086 **Effective Address (EA)** is a 16-bit binary offset added to a Segment Register to locate a byte in RAM.

2.2 Stage 2: 3-Second Visual Classifier ("Shakal Pehchano") & Decision Tree

```
MEMORY vs NO-MEMORY EFFECTIVE ADDRESS (EA) DECISION TREE
Instruction Operand
 +-- No Brackets [ ]?
 |    +-- Constant (#30) -----> 1. Immediate Mode (No memory EA)
 |    +-- Register (BX) ------> 2. Register Mode  (No memory EA)
 +-- Inside Brackets [ ... ]? (Accesses Memory -> Has an EA!)
      +-- [1234h] ------------> 3. Direct Mode (EA = 1234h)
      +-- [SI] or [BX] -------> 4. Register Indirect Mode (EA = [SI])
      +-- [BX + 5] -----------> 5. Based Mode (BX/BP + disp)
      +-- [SI + 5] -----------> 6. Indexed Mode (SI/DI + disp)
      +-- [BX][SI] -----------> 7. Based Indexed Mode (BX + SI)
      +-- [BX][SI + 20] ------> 8. Based Indexed with Displacement (BX + SI + 20)
```

2.3 Stage 3: Exact Exam-Sheet Write-Up (Approved M2 Q1 & ESE 2023)

Definitions:

- **Addressing Mode:** Specifies the rule for interpreting or modifying the address field of an instruction before the operand is actually referenced.
- **Effective Address (EA):** The offset calculated from the selected addressing mode that gives the actual memory location of the operand.
- **Base Registers:** BX (Base Register) or BP (Base Pointer). | **Index Registers:** SI (Source Index) or DI (Destination Index).

#	Addressing Mode	How to Recognize & Effective Address (EA) Formula	Example from Notes
1	**Immediate**	Constant value is inside the instruction itself. (EA = None)	`MOV CL, #30`
2	**Register**	Operand is in a named CPU register. (EA = None)	`MOV AX, BX`
3	**Direct**	Memory address (offset) is written directly in [ ]. (EA $= 1234$h)	`ADD AX, [1234h]`
4	**Register Indirect**	A register holds the memory offset inside [ ]. (EA $= [\text{SI}]$)	`MOV [SI], AL`
5	**Based**	Base register (BX/BP) + displacement. (EA $= \text{BX} + 5$)	`MOV AL, [BX+5]`
6	**Indexed**	Index register (SI/DI) + displacement. (EA $= \text{SI} + 5$)	`MOV AL, [SI+5]`
7	**Based Indexed**	Base register + Index register. (EA $= \text{BX} + \text{SI}$)	`ADD AX, [BX][SI]`
8	**Based Indexed + Disp**	Base + Index + constant displacement. (EA $= \text{BX} + \text{SI} + 20$)	`ADD AX, [BX][SI+20]`

Mandatory Exam Justification Wording: *"`MOV AL, [BX+5]` uses **based addressing** because the effective address is obtained from base register* BX *plus displacement* 5*."*

[SILLY-DOUBT CLEARED] How do I never confuse Based ([BX+5]) with Indexed ([SI+5])?
Answer: Look at the letters! **BX / BP = Based**. **SI / DI (Index registers) = Indexed**!

Subtopic 2.1B: Instruction Formats [Textbook-Filled for Syllabus 2.1]

An **Instruction Format** defines the bit layout of an instruction ([Opcode | Address Field(s)]). Based on the number of explicit operand addresses, processors use 4 formats:

Format Type	Structure	Example Operation	Key Hardware Trade-Off
3-Address	`OP Dest, Src1, Src2`	`ADD R1, A, B` ($R_1 \leftarrow M[A] + M[B]$)	Shortest programs (fewest instructions), but each instruction is very long in bits.
2-Address	`OP Dest/Src1, Src2`	`MOV R1, A`; `ADD R1, B` ($R_1 \leftarrow R_1 + M[B]$)	Most common in CISC (8086); overwrites one source operand so an extra `MOV` is needed.
1-Address	`OP Src` (Implicit AC)	`LOAD A`; `ADD B` (AC $\leftarrow$ AC $+ M[B]$)	Short instructions; relies on a single Accumulator (AC) register bottleneck.
0-Address	OP only (Uses Stack)	`PUSH A`; `PUSH B`; `ADD`	Tiny instructions (fast fetch); all operations pop/push the Top of Stack (TOS).

Subtopic 2.1C: Write Any 5 Instructions of 8086 Processor (ESE 2023 Q1v — 5 Marks)

1. **`MOV AX, BX` (Data Transfer — 1M):** Copies the 16-bit contents of source register BX into destination register AX (AX $\leftarrow$ BX).
2. **`ADD AX, [1234h]` (Arithmetic Addition — 1M):** Adds the 16-bit word at memory offset 1234h to AX and stores the sum in AX.
3. **`SUB BX, CX` (Arithmetic Subtraction — 1M):** Subtracts the contents of CX from BX and stores the result in BX (BX $\leftarrow$ BX $-$ CX).
4. **`INC SI` (Increment — 1M):** Adds 1 to the contents of index register `SI` (SI $\leftarrow$ SI $+ 1$).
5. **`JMP 2000h` (Unconditional Jump — 1M):** Transfers program execution directly to memory offset `2000h` by updating the Instruction Pointer (`IP/PC`).

2.4 Stage 4: Worked $\rightarrow$ Cold Drill (EA & Modes)

L1 Worked Example: Identify the addressing mode and compute EA for `MOV AL, [BX+SI+20]` given BX $=$ 1000h and SI $=$ 0200h.

- **Mode & Justification: Based Indexed with Displacement Addressing Mode** because the effective address is the sum of base register BX, index register `SI`, and displacement `20`.
- **Calculation (Show both Hex and Decimal interpretations of `20` for 100% safety):**
  - If `20` is hexadecimal (`20h`): EA $= 1000$h $+ 0200$h $+ 20$h $=$ **1220**h.
  - If `20` is decimal ($20_{10} = 14$h): EA $= 1000$h $+ 0200$h $+ 14$h $=$ **1214**h.

L3 Cold Drill: Identify the mode and write the 1-line EA justification for: (1) `MOV CL, #30`, (2) `MOV [SI], AL`, (3) `ADD AX, [BX][SI]`.

— COVER BELOW AND WRITE ON PAPER FIRST —

1. **Immediate Addressing:** Operand constant `30` is inside the instruction itself (no memory EA).
2. **Register Indirect Addressing:** Effective address (EA) of the destination memory location is held in register `SI`.

3. **Based Indexed Addressing:** Effective address (EA) is obtained by adding base register BX and index register SI (EA = BX + SI).

2.5 Stage 5: Contrast Table & 5-Line Survival Recall Card

Contrast Point	Register (`MOV AX, BX`)	Direct (`ADD AX, [1234h]`)	Register Indirect (`MOV [SI], AL`)
Where is the operand?	Inside CPU register BX	In Memory at offset 1234h	In Memory at the address held inside SI
Memory trips in Execute	0 (Fastest)	1	1 (Pointer register SI gives address)

3. Priority #3: Instruction Cycle, Interrupts, Micro-Operations & Memory Read/Write (Mod 2.1 | 10–15 Marks)

Mode: T + Chain | **Target:** Approved M2 Q3, Q4, Q5 + ESE 2023 Q1ii (5M) | **Time:** 10 Mins

3.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
When the CPU wants to fetch an instruction from address 100 in Main Memory, why can't the Program Counter (PC) just grab the instruction directly from RAM? Why must it pass through MAR and MBR?

— COVER BELOW AND WRITE ON PAPER FIRST —

[ANSWER]: Because PC is an internal pointer register, whereas only **MAR (Memory Address Register)** is physically wired to the external **Address Bus**, and only **MBR (Memory Buffer Register)** is physically wired to the external **Data Bus**!

Left Board (Dr. Berg's Diagnostic Whiteboard)

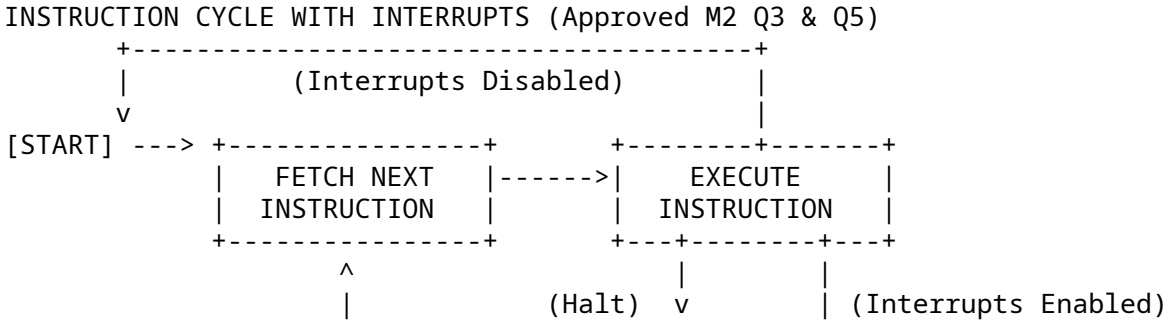
- **Symptom:** In the exam, students mix up which register moves into which during Fetch, Indirect, Execute, and Interrupt cycles.
- **Root Cause:** Memorizing register letters (MAR <- PC) as random alphabet soup instead of seeing their physical bus wiring.
- **Hardware Antidote:** Lock the **4 Core Registers (Approved M2 Q4)** to their exact physical wires:
  1. **PC (Program Counter):** Holds the address of the **next instruction** to be fetched.
  2. **MAR (Memory Address Register):** Holds the target memory address; **wired directly to the Address Bus**.
  3. **MBR / MDR (Memory Buffer / Data Register):** Holds data/instruction traveling in or out of memory; **wired directly to the Data Bus**.
  4. **IR (Instruction Register):** Holds the **current fetched instruction** so the Control Unit can decode its opcode.

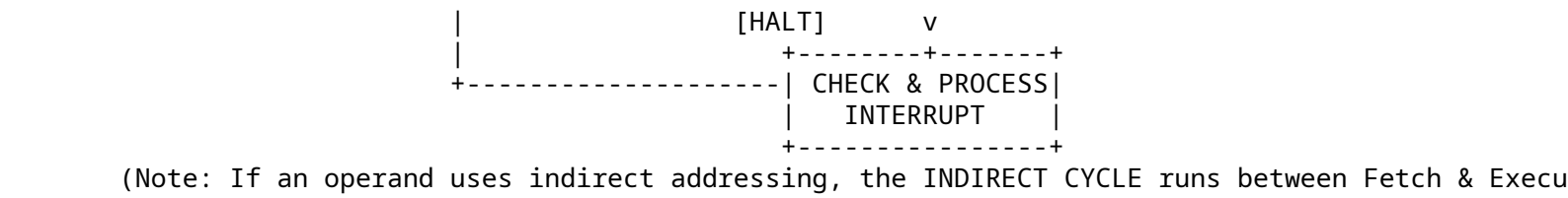
Right Board (Gaurav Thakur's Fetch Domino Relay & Kitchen/Pantry Map)

Watch the **Fetch Domino Relay** in Kitchen terms (Memory = Pantry, PC = Finger on recipe line, MAR = Megaphone shouting shelf number, MBR = Serving tray, IR = Eyes reading the recipe):

1. **Step 1 (MAR $\leftarrow$ PC):** Your finger (PC) tells the megaphone (MAR) which shelf number to shout.
2. **Step 2 (MBR $\leftarrow$ Memory[MAR]; PC $\leftarrow$ PC + 1):** The pantry puts that recipe card onto the serving tray (MBR), while your finger (PC) immediately slides down to the next line!
3. **Step 3 (IR $\leftarrow$ MBR):** The serving tray (MBR) hands the recipe card to your eyes (IR) to decode!
4. **Where the analogy breaks:** A kitchen tray carries physical paper, whereas MBR latches 16/32/40-bit electrical voltages from the system data bus on a clock edge.

3.2 Stage 2: Zero-Wrap Instruction Cycle with Interrupts Diagram





3.3 Stage 3: Exact Exam-Sheet Write-Up (Approved M2 Q3, Q4, Q5 & ESE 2023 Q1ii)

Part A: Overview of the 4 Stages of the Instruction Cycle

Cycle Stage	What Happens in Plain Words
1. Fetch Cycle	Reads the next instruction from Main Memory into the processor (IR) and increments PC.
2. Indirect Cycle	*(If needed)* Accesses memory to fetch the actual **Effective Address (EA)** of an indirect operand.
3. Execute Cycle	Interprets the opcode in IR and performs the required arithmetic, logic, data transfer, or branch operation.
4. Interrupt Cycle	After execution, checks for an enabled pending interrupt; saves the return PC to memory and jumps PC to the interrupt routine.

Part B: Register-Level Micro-Operations with "Because" Justifications

Each cycle consists of small register-transfer steps called **micro-operations**:

1. **Fetch Cycle Micro-Operations:**
   - t_1: MAR $\leftarrow$ (PC) — *Copy the next instruction address from* PC *into* MAR.
   - t_2: MBR $\leftarrow$ Memory[MAR] ; PC $\leftarrow$ (PC) + 1 — *Read the instruction from memory into* MBR*, and update* PC *to the next instruction.*
   - t_3: IR $\leftarrow$ (MBR) — *Place the fetched instruction into* IR *for decoding.*
2. **Indirect Cycle Micro-Operations:**
   - t_1: MAR $\leftarrow$ ($IR_{address}$) — *Copy the address field of* IR *(pointer address) into* MAR.
   - t_2: MBR $\leftarrow$ Memory[MAR] — *Read memory at that pointer address into* MBR *to get the real operand address.*
   - t_3: $IR_{address}$ $\leftarrow$ (MBR) — *Update the address field of* IR *with the real operand address obtained from* MBR.
3. **Execute Cycle Micro-Operations (ESE 2023 Q1ii — 5 Marks Standalone Question):**
   - The micro-operations in the Execute cycle depend on the opcode of the instruction.
   - **Standard Exam Example (`ADD R1, X` — Add contents of memory location X to register R_1):**
   - t_1: MAR $\leftarrow$ ($IR_{address}$) — *Place the operand memory address X from* IR *into* MAR.
   - t_2: MBR $\leftarrow$ Memory[MAR] — *Read the operand value from memory location X into* MBR.
   - t_3: $R_1 \leftarrow (R_1) + (\text{MBR})$ — *The ALU adds the contents of R_1 and* MBR*, keeping the result in R_1.*
4. **Interrupt Cycle Micro-Operations:**
   - t_1: MBR $\leftarrow$ (PC) — *Copy* PC *into* MBR *so the return address can be saved.*
   - t_2: MAR $\leftarrow$ Save_Address ; PC $\leftarrow$ Routine_Start_Address — *Load* MAR *with the memory save address and* PC *with the interrupt-routine starting address.*
   - t_3: Memory[MAR] $\leftarrow$ (MBR) — *Store* MBR *(containing the old* PC*) into memory.*

Part C: PC, IR, MAR, MBR/MDR & Memory Read/Write Sequencing (Approved M2 Q4)

Operation	Exact Hardware Sequence	Mandatory Sequencing Rule
Memory Read	**1.** Place required address in MAR. **2.** Issue the Read control signal. **3.** Memory value at that address is received in MBR/MDR.	Address must be placed in MAR before issuing the Read signal.
Memory Write	**1.** Place destination address in MAR **AND** data in MBR/MDR. **2.** Once **both** are ready, issue the `Write` control signal. **3.** Memory stores the data at the selected address.	**Both address (MAR) and data (MBR) must be ready BEFORE issuing the `Write` signal!**

[SILLY-DOUBT CLEARED] 1. Why can PC <- PC + 1 happen in the same step (t2) as MBR <- Memory[MAR]?
Answer: Because `MBR <- Memory[MAR]` uses the external System Bus, whereas `PC <- PC + 1` uses an internal incrementer inside the CPU. Since they do not compete for the same wires, they execute in parallel!

[SILLY-DOUBT CLEARED] 2. What happens if the Write signal is issued before loading MBR?
Answer: Memory immediately latches whatever garbage bits were sitting in MBR and overwrites the destination memory cell!

3.4 Stage 4: Active Retrieval Drills (L2 & L3)

L2 Faded Drill: Complete the Interrupt Cycle micro-operations: (1) t_1: [???] $\leftarrow$ (PC); (2) t_2: MAR $\leftarrow$ Save_Address; [???] $\leftarrow$ Routine_Start_Address; (3) t_3: Memory[MAR] $\leftarrow$ ([???]).

— COVER BELOW AND WRITE ON PAPER FIRST —

[SOLUTION]: (1) **MBR**, (2) **PC**, (3) **MBR**.

4. Priority #4: CPU Architecture, Internal Structure & Register Organization (Mod 2.1 | 10 Marks)

Mode: T (Theory) | **Target:** ESE 2023 Q2A OR ("Draw and explain internal structure of CPU" — 10M) + Syllabus 2.1 | **Time:** 7 Mins

4.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
You already know the external System Bus connects the CPU to Main Memory and I/O outside the chip. But inside the CPU chip itself, how do `Registers`, the ALU, and the `Control Unit` pass numbers to each other?

— COVER BELOW AND WRITE ON PAPER FIRST —

[ANSWER]: Through a dedicated high-speed internal highway inside the processor chip called the **Internal CPU Bus**, assisted by temporary holding registers around the ALU!

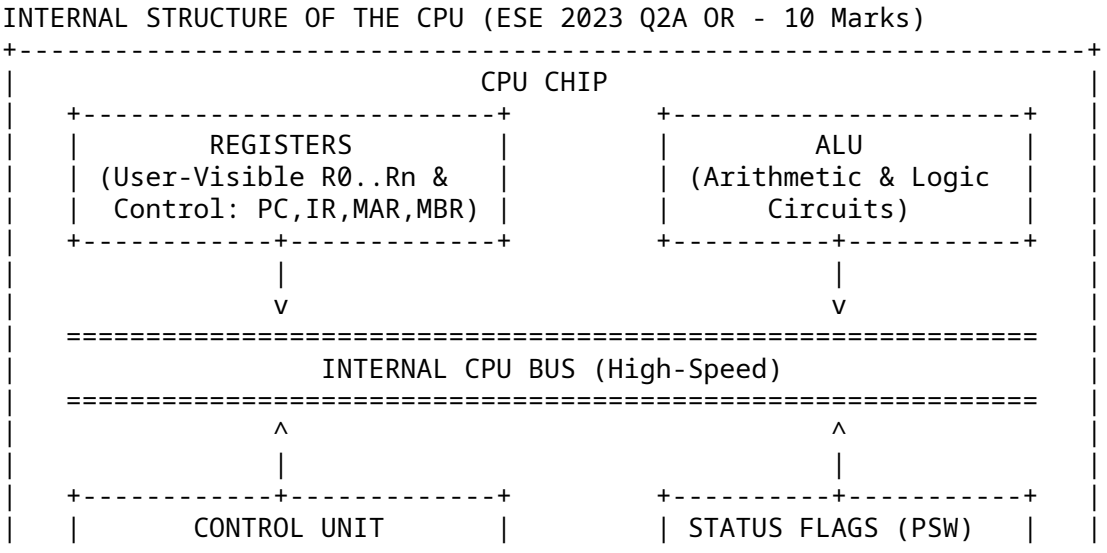
Left Board (Dr. Berg's Diagnostic Whiteboard)

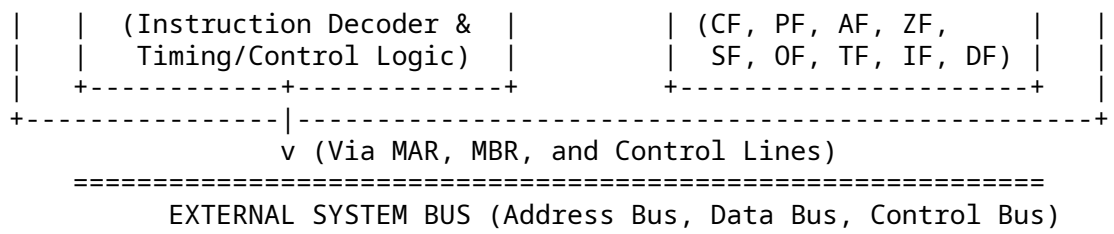
- **Symptom:** When asked *"Draw and explain the internal structure of the CPU (10M)"* in ESE 2023, students accidentally draw the outer Von Neumann computer block diagram instead of zooming **inside** the CPU box!
- **Root Cause:** Confusing the outer **Computer Structure** (CPU + Memory + I/O) with the **Internal Structure of the CPU**.
- **Hardware Antidote:** Zoom inside the CPU chip! It contains 5 internal elements: **ALU**, **Registers (User-Visible + Control/Status)**, **Control Unit**, **Status Flags**, and the **Internal CPU Bus**.

Right Board (Gaurav Thakur's Master Chef Countertop Analogy)

1. **Internal Countertop (`Internal CPU Bus`):** Inside the locked kitchen (**CPU**), the chef moves chopped veggies from small bowls (**Registers**) onto the chopping board (**ALU**) using his own arms (**Internal CPU Bus**).
2. **Pass-Through Window (`MAR` & `MBR`):** Only when the chef needs raw ingredients from the outer Warehouse (**Main Memory**) does he shout through `MAR` (Address Bus) and receive trays at `MBR` (Data Bus).
3. **Where the analogy breaks:** Kitchen bowls sit passively on a table, whereas CPU registers are clocked D-flip-flop arrays that latch binary bits in nanoseconds when enabled by Control Unit lines.

4.2 Stage 2: Zero-Wrap Internal Structure of the CPU Diagram (10M)





4.3 Stage 3: Exact Exam-Sheet Write-Up (10 Marks)

Part A: Five Internal Components of the CPU (ESE 2023 Q2A OR)

Internal CPU Component	Role & Function Inside the Processor
1. Arithmetic Logic Unit (ALU)	Performs arithmetic (`ADD`, `SUB`, `MUL`, `DIV`) and logical (`AND`, `OR`, `NOT`, `XOR`, `SHIFT`) operations on binary data supplied by registers.
2. Control Unit (CU)	Interprets instructions from `IR` and generates timing and control signals that govern data movement across the Internal CPU Bus and External System Bus.
3. CPU Registers	Provides ultra-fast local storage inside the CPU for instructions, addresses, and operands.
4. Status Flags Register (PSW)	Records status conditions produced by the ALU (`CF`, `PF`, `AF`, `ZF`, `SF`, `OF`) and holds control bits (`TF`, `IF`, `DF`).
5. Internal CPU Bus	Internal data/control pathway connecting Registers, ALU, and Control Unit to each other and to the external System Bus interface (`MAR` and `MBR`).

Part B: Register Organization in a CPU [Syllabus 2.1 & Approved M2 Q4]

CPU registers are organized into two functional categories:

1. **User-Visible Registers** *(Accessible to assembly programmers to minimize main-memory references):*
   - **General-Purpose Registers (`AX`, `BX`, `CX`, `DX` or `R0..Rn`):** Hold operands and intermediate results for arithmetic/logic operations.
   - **Data Registers:** Used strictly for holding data values (cannot be used in effective address calculation).
   - **Address Registers:** Hold memory addresses or offsets: **Index Registers (`SI`, `DI`)** for indexed addressing, **Segment Pointers (`CS`, `DS`, `SS`, `ES`)** for base segment addresses, and **Stack Pointer (`SP`, `BP`)** for stack operations (PUSH/POP).
2. **Control and Status Registers** *(Used by the Control Unit to control processor execution — Approved M2 Q4):*
   - **PC (Program Counter):** Holds the address of the **next instruction** to be fetched.
   - **`IR` (Instruction Register):** Holds the **current/most recently fetched instruction**.
   - **`MAR` (Memory Address Register):** Holds the memory address for a read/write; connected to the **Address Bus**.
   - **`MBR` / `MDR` (Memory Buffer / Data Register):** Holds data read from or written to memory; connected to the **Data Bus**.
   - **`PSW` / Flag Register:** Holds condition codes (Status Flags) and execution mode bits (Control Flags).

4.4 Stage 4: 5-Line Survival Recall Card

1. Name the 5 internal blocks inside the CPU chip. → **ALU, Control Unit, Registers, Status Flags, Internal CPU Bus.**
2. What are the two main categories of registers in Register Organization? → **User-Visible Registers and Control & Status Registers.**
3. Name the 4 essential Control & Status registers used during the instruction cycle. → **`PC`, `IR`, `MAR`, `MBR`/`MDR`.**
4. Name the 3 types of Address Registers under User-Visible Registers. → **Index Registers (`SI`/`DI`), Segment Pointers (`CS`/`DS`/`SS`/`ES`), Stack Pointer (`SP`).**

5. Which internal bus connects the ALU and Registers inside the processor? → **The Internal CPU Bus.**

5. Priority #5: Need of Control Unit, Wilkes' Design & Hardwired vs Microprogrammed (Mod 2.2 | 15 Marks)

Mode: T + C | **Target:** ESE 2023 Q2Bii (10M) + ESE 2023 Q5i (5M) [Textbook-Filled Syllabus 2.2] | **Time:** 10 Mins

5.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
The ALU knows how to add, and Registers know how to hold bits. Why would the entire CPU sit completely frozen without a **Control Unit**? And what are the two opposite ways engineers build that Control Unit?

— COVER BELOW AND WRITE ON PAPER FIRST —

[ANSWER]: Registers and the ALU have electronic "gates" (switches) blocking their inputs and outputs. The **Control Unit** is the conductor that opens the exact right gates in the right clock step (t_1, t_2, t_3). You can build it either by **soldering fixed logic gates (Hardwired)** or by **storing a lookup recipe in a small ROM inside the CPU (Micro-programmed)**!

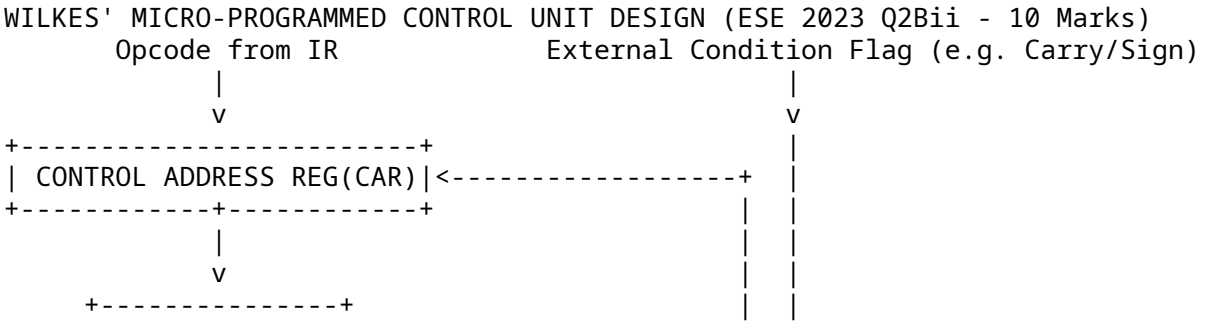
Left Board (Dr. Berg's Diagnostic Whiteboard)

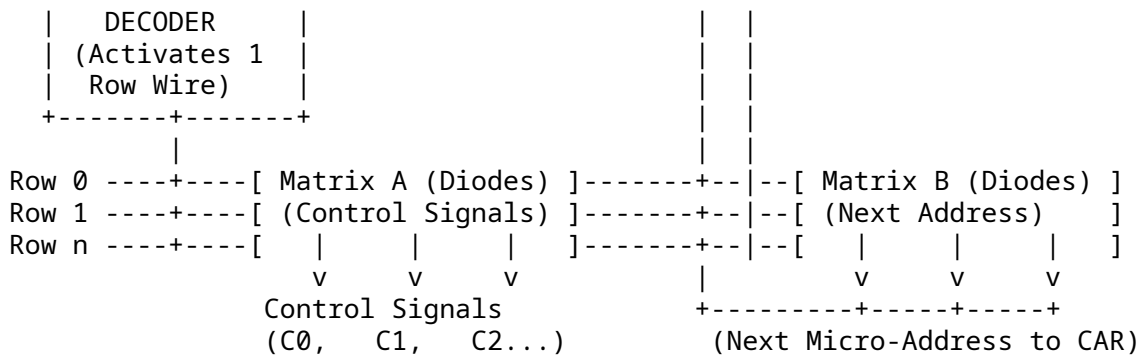
- **Symptom:** Designing a Control Unit for 200 complex instructions using raw `AND`/`OR`/`NOT` logic gates creates a giant, tangled "spaghetti circuit" that is impossible to fix if a bug is found.
- **Root Cause: Hardwired Control** wires every control signal directly out of fixed boolean logic circuits. Fast, yes — but zero flexibility!
- **Hardware Antidote:** In 1951, **Maurice Wilkes** invented **Micro-programmed Control**. Instead of soldering gates for every step, store the control signals (`101001...`) as rows ("micro-instructions") inside a tiny fast ROM (**Control Memory**) inside the CPU!

Right Board (Gaurav Thakur's Music Box vs Playlist Analogy)

1. **Hardwired Control = Mechanical Music Box with Metal Pins:** Super fast for 1 song, but if you want to change the tune, you must melt the metal cylinder and rebuild it from scratch!
2. **Micro-programmed Control = Playlist stored on a Pen-Drive (Control Memory ROM):** Each line of the playlist (**Micro-instruction**) turns speakers ON/OFF (**Control Signals**) and tells the player which line to play next (**Next Address**)!
3. **How Wilkes' Diode Matrix Works:** Wilkes used horizontal row wires (1 row per micro-instruction) intersecting two sets of vertical column wires via diodes:
   - **Left Grid (Matrix A):** Vertical wires go out to the CPU as **Control Signals** ($C_0, C_1 \ldots$).
   - **Right Grid (Matrix B):** Vertical wires feed back into the **Control Address Register (CAR)** to pick the **Next Micro-instruction Address**!
4. **Where the analogy breaks:** A music playlist plays audio beats over seconds, whereas Wilkes' diode matrix fires electrical control voltages (`MAR <- PC, Read`) every single clock cycle (nanoseconds).

5.2 Stage 2: Zero-Wrap Wilkes' Micro-Programmed Control Unit Diagram (10M)





5.3 Stage 3: Exact Exam-Sheet Write-Up (10M + 5M)

Part A: What is the Need of a Control Unit? (ESE 2023 Q2Bii Part 1 — 3 Marks)

The **Control Unit (CU)** is required for three fundamental operations inside the processor:

1. **Instruction Interpretation (Decoding):** Reads the opcode from the Instruction Register (`IR`) and decodes what operation is required.
2. **Instruction Sequencing:** Steps the CPU through the correct chronological sequence of micro-operations ($t_1, t_2, t_3 \dots$) across Fetch, Indirect, Execute, and Interrupt cycles.
3. **Control Signal Generation:** Issues electrical control signals to open/close register gates onto the Internal CPU Bus, select ALU operations (`ADD/SUB`), and assert external Control Bus commands (`Memory Read/Write`).

Part B: Wilkes' Micro-Programmed Control Unit in Detail (ESE 2023 Q2Bii Part 2 — 7 Marks)

Proposed by **Maurice Wilkes (1951)**, this design replaces complex random logic gates with an orderly **diode matrix ROM array**:

1. **Two-Matrix Diode Array Structure:** Consists of horizontal row wires (each row represents one **micro-instruction**) intersecting two groups of vertical column wires coupled by diodes:
   - **Matrix A (Control Signal Generator):** Vertical lines output the individual **Control Signals** ($C_0, C_1, C_2 \dots$) to the CPU data path. Wherever a diode connects an active row to a column, that control signal is asserted (1).
   - **Matrix B (Next-Address Generator):** Vertical lines generate the **address of the next micro-instruction** and feed it back into the Control Address Register (CAR).
2. **Control Address Register (CAR):** Holds the address of the current micro-instruction row. At the start of an Execute cycle, the **Opcode** from `IR` is loaded into CAR.
3. **Address Decoder:** Decodes the n-bit address in CAR so that **exactly one horizontal row wire** is energized per clock cycle.
4. **Conditional Branching:** A condition flag from the ALU (such as Carry or Sign) controls a two-way switch on selected rows before entering Matrix B, allowing the micro-program to branch to one of two different next addresses based on the flag value (0 or 1).

Part C: Hardwired vs Micro-Programmed Control Unit (ESE 2023 Q5i — 5 Marks)

Parameter	Hardwired Control Unit	Micro-Programmed Control Unit
1. Implementation	Implemented using **fixed hardware logic circuits** (gates, flip-flops, decoders, counters).	Implemented using a **Control Memory (ROM)** storing micro-instructions (control words).
2. Speed of Operation	**Very fast** (signals propagate at logic-gate speed; no control-memory access delay).	**Comparatively slower** (each step requires reading a micro-instruction from Control Memory).
3. Flexibility	**Difficult to modify**; changing an instruction requires hardware rewiring.	**Highly flexible**; instructions are modified by updating the microprogram in ROM.
4. Complex Instructions	**Difficult and expensive** to design for large, complex instruction sets.	**Easy and systematic** to handle complex instruction sets via micro-routines.
5. Design & Debugging	Complex, time-consuming, and error-prone layout.	Orderly, modular, and simpler to debug and maintain.
6. Processor Type	Used in **RISC** architectures.	Used in **CISC** architectures.

5.4 Stage 4: 5-Line Survival Recall Card

1. What are the 3 core needs of a Control Unit? → **Decoding instructions, Sequencing micro-operations, Generating control signals.**
2. In Wilkes' design, what do Matrix A and Matrix B generate? → **Matrix A = Control Signals; Matrix B = Next Micro-instruction Address.**
3. What is the role of CAR and the Decoder in Wilkes' unit? → **CAR holds current micro-address; Decoder activates 1 row wire.**
4. Which control unit is faster (Hardwired vs Microprogrammed) and why? → **Hardwired (no Control Memory ROM fetch delay).**
5. Which architecture (RISC vs CISC) uses Hardwired vs Microprogrammed control? → **RISC = Hardwired; CISC = Microprogrammed.**

6. Priority #6: Horizontal vs Vertical, Sequencing, Applications & Nano-Programming (Mod 2.2 | 5–10 Marks)

Mode: T + C | **Target:** ESE 2023 Q2Aii (5M) + Syllabus 2.2 [Textbook-Filled] | **Time:** 7 Mins

6.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
Suppose your CPU has 32 internal control switches ($C_0 \dots C_{31}$). In a Micro-programmed Control Unit, should each micro-instruction store **32 separate bits** (1 bit per switch), or a compact **5-bit binary code** ($2^5 = 32$) feeding a decoder?

— COVER BELOW AND WRITE ON PAPER FIRST —

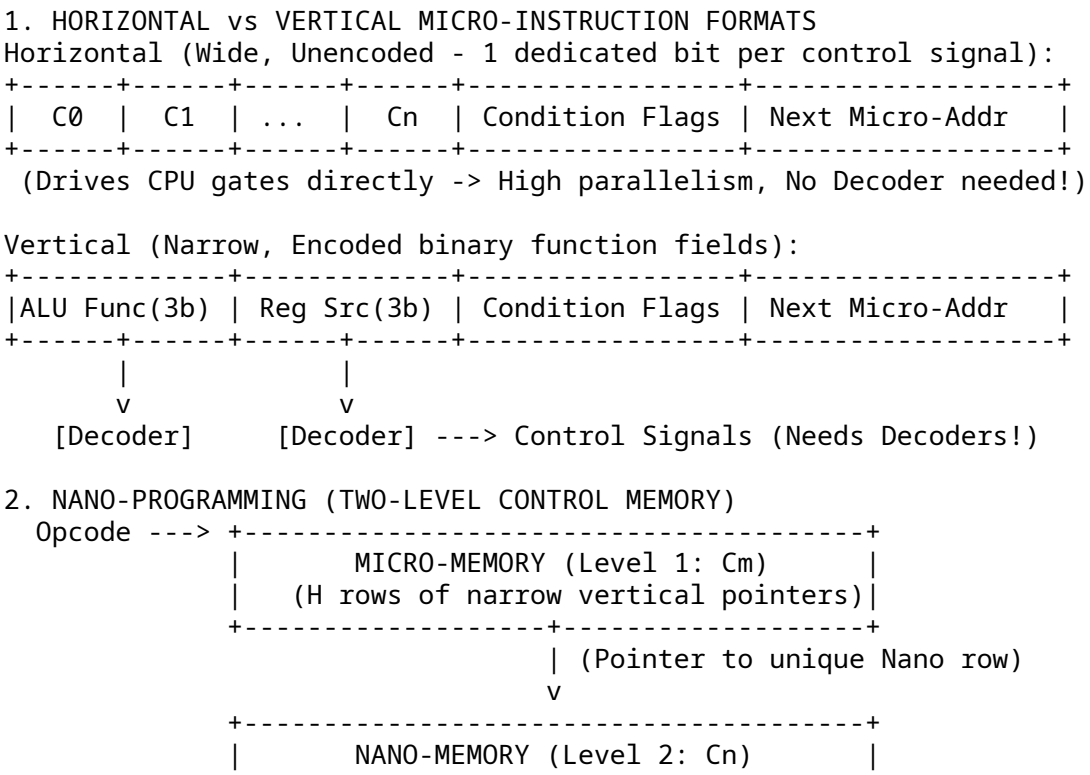
[ANSWER]: That exact trade-off is **Horizontal vs Vertical Micro-instructions**!

- **Horizontal = 1 bit per control signal** (wide word, no decoder needed, many switches turn ON in parallel).
- **Vertical = Encoded binary fields** (narrow word, needs a decoder, saves ROM width but runs slightly slower).

Left Board (Dr. Berg's Diagnostic Whiteboard) & Right Board (Gaurav Thakur's Analogy)

- **Horizontal vs Vertical Analogy: Horizontal** is a 32-switch wall panel where you can flip 10 switches simultaneously with both hands (zero decoder delay). **Vertical** is typing "Code 14" on a keypad — compact, but a decoder box must translate "14" before one switch turns on!
- **Nano-Programming (Two-Level Control Store):** Suppose a 100-bit wide micro-instruction pattern is repeated 20 times across different instructions. Instead of storing all 100 bits 20 times in one giant ROM, store **narrow pointers** in a top **Micro-Memory (C_M)** that point to a small bottom **Nano-Memory (C_N)** holding only the *unique* 100-bit control words!
- **Where the analogy breaks:** A songbook chorus pointer saves paper ink, whereas Nano-programming saves physical silicon area (bits $=$ rows $\times$ width) inside a two-level CPU ROM hierarchy.

6.2 Stage 2: Zero-Wrap Diagrams (Horizontal/Vertical & Nano-Programming)



```
|  (Only n unique wide horizontal words)|
+-------------------+-------------------+
                    |
                    v
        CPU Control Signals (W bits wide)
```

6.3 Stage 3: Exact Exam-Sheet Write-Up (5M / 10M)

Part A: Horizontal vs Vertical Micro-Instructions (5 Marks)

Parameter	Horizontal Micro-Instructions	Vertical Micro-Instructions
1. Bit Encoding	**Unencoded** (1 dedicated bit per control signal).	**Highly encoded** (binary code fields).
2. Word Width	**Long / Wide** (40 to $100+$ bits per word).	**Short / Narrow** (16 to 32 bits per word).
3. Decoders Needed	**No decoders required** (bits drive lines directly).	**Requires hardware decoders** (k bits $\rightarrow 2^k$ lines).
4. Parallelism	**High parallelism** (many signals fire simultaneously).	**Limited parallelism** (1 signal per decoded field).
5. Execution Speed	**Faster** (zero decoder propagation delay).	**Slower** (incurs decoder delay).
6. Memory Width	Requires wider Control Memory words.	More compact Control Memory width.

Part B: Micro-Instruction Sequencing and Execution (5 Marks)

1. **Micro-Instruction Sequencing (Selecting the Next Address in CAR):** The **Micro-program Sequencer** determines the next address loaded into the **Control Address Register (CAR)** using one of three sources: (a) **Next sequential address** (CAR $\leftarrow$ CAR $+ 1$), (b) **Branch address** from the micro-instruction when a status flag condition is met, or (c) **Opcode mapping** from IR at the start of an instruction.
   - *Three Addressing Techniques:* **(i) Two-Address Fields** (both branch addresses stored in the micro-instruction), **(ii) Single-Address Field** (CAR $+ 1$ for sequential, address field for branch), and **(iii) Variable Format** (one bit switches the word between control-signal format and branch format).
2. **Micro-Instruction Execution (Generating Control Signals):** The address in CAR reads the micro-instruction from **Control Memory (ROM)** into the **Control Buffer Register (CBR)**. The bits in CBR directly (or via decoders) open internal CPU register gates and trigger the ALU for that clock cycle.

Part C: Applications of Microprogramming (ESE 2023 Q2Aii — 5 Marks)

1. **Realization of Complex Control Units (CISC):** Makes designing and modifying control units for large instruction sets (like Intel x86 and IBM System/370) systematic and error-free.
2. **Emulation of Other Computers:** Allows a host computer to execute machine programs written for a different target architecture by loading an emulator micro-program into Control Memory.
3. **Operating System Support:** Frequently used OS routines (process scheduling, context switching, interrupt handling) can be implemented in fast microcode.
4. **High-Level Language Support:** Complex high-level operations (array bounds checking, string manipulation, floating-point math) can be accelerated as custom micro-routines.
5. **Micro-Diagnostics & Field Bug Patching:** Built-in micro-diagnostic routines can test internal CPU hardware, and processor bugs can be fixed post-manufacture via firmware/microcode updates.

Part D: Concepts of Nano-Programming (5 Marks)

- **Core Idea:** When a wide Horizontal Control Memory (H words $\times W$ bits) has many duplicate control words and only n unique patterns ($n \ll H$), a **two-level control store** saves massive ROM area:
  1. **Level 1 — Micro-Memory (C_M):** Holds H narrow pointers of width $\lceil \log_2 n \rceil$ bits.
  2. **Level 2 — Nano-Memory (C_N):** Holds only the n unique wide control words of width W bits.

- **Memory Saving Formula & Numerical Example:**

$$\text{Single-Level ROM} = H \times W \quad \text{vs} \quad \text{Nano-Programmed ROM} = (H \times \lceil \log_2 n \rceil) + (n \times W)$$

For $H = 1024$ rows, $W = 100$ bits, and $n = 64$ unique words ($\log_2 64 = 6$ bits): Single-Level $= 1024 \times 100 =$ **102**, **400** bits, whereas Nano-Programmed $= (1024 \times 6) + (64 \times 100) =$ **12**, **544** bits (**saves** 87.7% **of ROM bits!**).

7. Priority #7: CISC vs RISC, RISC Pipelining & Hazards (Mod 2.2 | 5–10 Marks)

Mode: T + C | **Target:** ESE 2023 Q2Ai (5M) + ESE 2023 Q5v (5M) + Syllabus 2.2 [Textbook-Filled] | **Time:** 6 Mins

7.1 Stage 1: Understand the Concept First (Simple Vocabulary & Analogy)

[HOOK QUESTION] (Think for 5 seconds before looking below):
Would a restaurant serve customers faster if it had a 300-page menu of complicated dishes where the chef runs to the distant supermarket for every single ingredient (**CISC**), OR a 10-item menu where every ingredient is kept right on the kitchen counter and 3 chefs work like an assembly line (**RISC + Pipelining**)?

— COVER BELOW AND WRITE ON PAPER FIRST —

[ANSWER]: The 10-item assembly-line kitchen (**RISC with Pipelining**)! Studies showed that 80% of program execution uses only 20% of the simplest instructions (`LOAD`, `STORE`, `ADD`, `BRANCH`).

Left Board (Dr. Berg's Whiteboard) & Right Board (Gaurav Thakur's Laundry Assembly Line)

- **Why CISC Struggles to Pipeline:** CISC instructions have variable lengths (1 to 15 bytes) and take variable clock cycles (1 to 20 cycles), causing stages to jam.
- **Why RISC Makes Pipelining Easy:** Every RISC instruction is **fixed-length** (32 bits), operates **Register-to-Register** in 1 cycle (only LOAD and STORE touch memory), so **Instruction Fetch (I)**, **Execute (E)**, and **Memory (D)** overlap like a smooth laundry assembly line (Washer $\rightarrow$ Dryer $\rightarrow$ Folder)!
- **Where the analogy breaks:** Laundry takes minutes per load, whereas a RISC pipeline overlaps hardware instruction stages across inter-stage latches every clock nanosecond.

7.2 Stage 2: Zero-Wrap RISC Pipelining Timing Diagram (ESE 2023 Q2Ai — 5M)

```
RISC PIPELINING TIMING DIAGRAM (I = Instruction Fetch, E = Execute, D = Memory Data)
1. Basic 2-Stage Pipeline (I and E overlapped):
Clock Cycle:     1       2       3       4       5
Instruction 1: [ I ]   [ E ]
Instruction 2:         [ I ]   [ E ]
Instruction 3:                 [ I ]   [ E ]
Instruction 4:                         [ I ]   [ E ]

2. 3-Stage RISC Pipeline (I, E, D overlapped):
Clock Cycle:     1       2       3       4       5
Instruction 1: [ I ]   [ E ]   [ D ]
Instruction 2:         [ I ]   [ E ]   [ D ]
Instruction 3:                 [ I ]   [ E ]   [ D ]
(Once full at Cycle 3, 1 instruction completes EVERY single clock cycle -> CPI = 1!)
```

7.3 Stage 3: Exact Exam-Sheet Write-Up (CISC vs RISC, Pipelining & Hazards)

Part A: Comparison Between CISC and RISC Architectures (Syllabus 2.2 — 5M / 10M)

Feature / Parameter	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
1. Instruction Count	**Large** set of complex instructions (100–250+).	**Small / Reduced** set of simple instructions (< 100).
2. Instruction Format	**Variable-length** format (1–15 bytes).	**Fixed-length**, uniform format (32-bit word).
3. Cycles / Instr (CPI)	Multi-cycle execution (2–15+ cycles per instruction).	Single-cycle execution (CPI ≈ 1).
4. Control Unit Design	**Micro-programmed** Control Unit (ROM control store).	**Hardwired** Control Unit for maximum speed.
5. Memory Access	Arithmetic/logic instructions can directly access Memory.	Strictly **LOAD/STORE architecture** (all ALU ops are Register-to-Register).
6. CPU Registers	**Few** general-purpose registers (8–16).	**Large register file** (32–192+ registers).
7. Addressing Modes	**Many complex** addressing modes (8–20+).	**Few simple** addressing modes (1–4).
8. Pipelining	**Difficult to pipeline** due to variable instruction durations.	**Highly optimized for pipelining**.
9. Processor Examples	Intel x86 (8086, Core), IBM System/370, VAX.	ARM, MIPS, RISC-V, SPARC.

Part B: What is RISC Pipelining? (ESE 2023 Q2Ai — 5 Marks)

1. **Definition: Instruction Pipelining** is an execution technique that overlaps multiple instructions in time by dividing the instruction cycle into independent sequential stages.
2. **Three Core RISC Pipeline Stages:**
   - **I (Instruction Fetch):** Fetches the next instruction from memory into the CPU.
   - **E (Execute):** Performs the register-to-register ALU operation or computes the memory address for LOAD/STORE.
   - **D (Memory Data):** Transfers data between a register and memory (only active during LOAD or STORE).
3. **Handling Stalls (NO-OP & Delayed Branch):** To prevent memory conflicts or branch penalties, RISC compilers insert a **NO-OP** bubble or use **Delayed Branching** (placing a useful independent instruction in the branch delay slot right after the jump so it executes while the branch target is fetched).

Part C: Bonus Short Note — Pipeline Hazards (ESE 2023 Q5v — 5 Marks)

A **Pipeline Hazard** is any condition that prevents the next instruction in the stream from executing during its designated clock cycle, causing a pipeline stall:

1. **Structural (Resource) Hazard:** Occurs when two instructions in different stages need the **same hardware resource** (e.g., single-port memory) during the same clock cycle.
2. **Data Hazard (RAW, WAR, WAW):** Occurs when an instruction depends on the **unwritten result of a preceding instruction** still in the pipeline (e.g., ADD R1, R2, R3 followed immediately by SUB R4, R1, R5).
3. **Control (Branch) Hazard:** Occurs during **conditional branches/jumps** when the pipeline fetches the wrong sequential instruction before the branch outcome is known.

8. Interleaved Boss Test & Quadruple-Check Syllabus/PYQ Audit

8.1 Part 1: R5 Interleaved Boss Set (3 Mixed Exam Problems)

1. **Boss Problem 1 (10M Archetype — 8086 Flag Decode):** *Shakal Pehchano:* 16-bit string with X and `0/1` → **8086 Flag Decoder (Bit 15 Left to Bit 0 Right)**.
   Decode bits `11, 10, 7, 6, 2, 0` and X in: `X X X X 1 1 0 0 1 0 X 0 X 1 X 1`.
2. **Boss Problem 2 (10M Archetype — Addressing Mode & Effective Address):** *Shakal Pehchano:* Brackets `[ ]` with registers and constants → **8086 Addressing Mode Classifier + EA**.
   Name the addressing mode, justify it, and compute EA (given BX $= 2000$h, SI $= 0500$h) for: (a) `MOV AL, [BX+10h]` and (b) `ADD AX, [BX][SI+05h]`.
3. **Boss Problem 3 (10M Archetype — RTL Micro-Ops & Control Unit Contrast):** (a) Write the 3 micro-operations of the **Indirect Cycle**. (b) In Wilkes' Control Unit, what is the difference between **Matrix A** and **Matrix B**?

— COVER BELOW AND WRITE ON PAPER FIRST —

[BOSS SET SOLUTIONS & SELF-SCORING RUBRIC]:

- **Boss Problem 1:** `Bit 11 (OF=1)`: Signed overflow occurred; `Bit 10 (DF=1)`: String auto-decrement; `Bit 7 (SF=1)`: Negative result; `Bit 6 (ZF=0)`: Non-zero result; `Bit 2 (PF=1)`: Even parity; `Bit 0 (CF=1)`: Carry out of MSB; `X (Bits 1,3,5,12..15)`: Undefined/reserved bits.
- **Boss Problem 2:** (a) `MOV AL, [BX+10h]` uses **Based Addressing** because EA $=$ BX $+ 10$h $= 2000$h $+ 10$h $=$ **2010**h. (b) `ADD AX, [BX][SI+05h]` uses **Based Indexed with Displacement Addressing** because EA $=$ BX $+$ SI $+ 05$h $= 2000$h $+ 0500$h $+ 05$h $=$ **2505**h.
- **Boss Problem 3:** (a) Indirect Cycle: (1) MAR $\leftarrow$ (IR$_{\text{address}}$), (2) MBR $\leftarrow$ Memory[MAR], (3) IR$_{\text{address}}$ $\leftarrow$ (MBR). (b) **Matrix A** generates the **Control Signals ($C_0, C_1 \ldots$)** for the CPU; **Matrix B** generates the **Next Micro-Instruction Address** fed back to CAR.

8.2 Part 2: Quadruple-Check Syllabus & PYQ Audit Matrix (Nothing Missed!)

Every single item from your official **Module 2.1 & 2.2 Syllabus**, **Approved Module 2 Questions (M2 Q1--Q5)**, and **Somaiya MSE 2025 & ESE 2023 Exam Papers** is verified 4 times in this document:

Official Syllabus / PYQ Topic	Exam / Syllabus Source	Exact Section in Th
1. 8086 16-bit Flag Register & Bit Decoding	MSE 2025 Q3 (10M), M2 Q2	**Section 1** (Pages 2
2. All 8 Addressing Modes & Effective Address (EA)	ESE 2023 Q2Bi (10M), M2 Q1	**Section 2** (Pages 4
3. Instruction Formats (`0, 1, 2, 3-Address`)	Syllabus 2.1	**Section 2, Sub 2.1**
4. Write Any 5 Instructions of 8086 Processor	ESE 2023 Q1v (5M)	**Section 2, Sub 2.1**
5. Basic Instruction Cycle & Interrupts (`Fetch..Interrupt`)	Approved M2 Q3 & Q5	**Section 3, Part A**
6. List the Micro-operations in Execute Cycle	ESE 2023 Q1ii (5M)	**Section 3, Part B**
7. Instruction Interpretation and Sequencing	Syllabus 2.1	**Section 3 & Sectio**
8. `PC, IR, MAR, MBR/MDR` & Read/Write Sequencing	Approved M2 Q4, 2020 MCQ	**Section 3, Part C**
9. CPU Architecture / Internal Structure of CPU	Syllabus 2.1, ESE 2023 Q2A OR	**Section 4, Part A**
10. Register Organization (User-Visible vs Control/Status)	Syllabus 2.1	**Section 4, Part B**
11. Need of Control Unit + Wilkes' Microprogrammed CU	Syllabus 2.2, ESE 2023 Q2Bii	**Section 5, Part A**
12. Hardwired vs Microprogrammed Control Unit	Syllabus 2.2, ESE 2023 Q5i	**Section 5, Part C**
13. Horizontal and Vertical Micro-Instructions	Syllabus 2.2	**Section 6, Part A**
14. Micro-instruction Sequencing and Execution	Syllabus 2.2	**Section 6, Part B**
15. Applications of Microprogramming	ESE 2023 Q2Aii (5M)	**Section 6, Part C**
16. Concepts of Nano-programming	Syllabus 2.2	**Section 6, Part D**
17. Comparison Between CISC and RISC Architectures	Syllabus 2.2	**Section 7, Part A**
18. RISC Pipelining (+ Pipeline Hazards Bonus)	ESE 2023 Q2Ai & Q5v (5M)	**Section 7, Part B**